Can LLM Coding Agents Reason About Time Series?

Filip Rechtorík **Ondřej Dušek** **Zdeněk Kasner**
Charles University, Faculty of Mathematics and Physics
Institute of Formal and Applied Linguistics
Prague, Czechia
{surname}@ufal.mff.cuni.cz

Abstract

Large language models (LLMs) are increasingly being used for automated decision-making systems in finance, healthcare, or environmental monitoring. Time series data are ubiquitous in these fields, yet hard to process automatically. Can time series be analyzed by LLM agents? We examine three approaches: providing the agent with raw numerical data, using the LLM as a coding agent, or a combination of both. In the coding agent setup, the model iteratively queries the data using Python code. Using two time series understanding benchmarks, we show that agents with code access can outperform models processing raw data by up to 10%. However, even the best performing agent still answers about 22–34% of the questions incorrectly. To get insights into models' strategies and reasoning gaps, we analyze the model outputs with a strong LLM judge. Our analysis reveals that coding agents can select appropriate statistical tests, but often miss important nuances. Meanwhile, models with access to raw data can reach the right conclusions using back-of-the-envelope calculations.[1]

1 Introduction

Analyzing and interpreting time series data is critical for making informed decisions. Tools for automated time series analysis are developed in many domains, the most prominent being finance (Tsay, 2005; Cipra, 2020), healthcare (Portet et al., 2009; Alsheheri, 2025), and environmental monitoring (Han et al., 2026; Jafari et al., 2024). However, time series analysis is still typically handled by human experts in the domain that can combine the output of the tools with domain knowledge and external context (Pirolli and Card, 2005; Imani et al., 2019; Holstein, 2024).

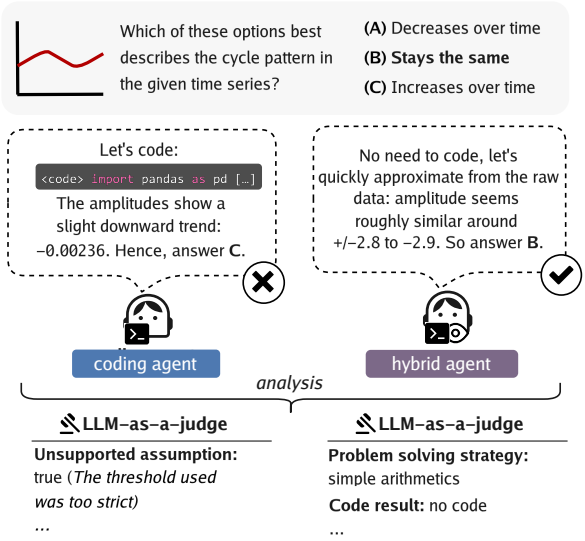


Figure 1: LLMs adapt their approaches to analyzing time series based on available data and tools. While using code as an intermediate step is more interpretable and accurate overall, coding agents can overrely on the code results (left side). At the same time, simple calculations on raw data can lead to correct conclusions, bypassing code entirely (right side).

Recently, there has been a surge of interest in using large language models (LLMs) to automate advanced engineering tasks, including time series analysis (Jin et al., 2025; Zhang et al., 2024; Heesch et al., 2026). The good performance of LLMs in various tasks across many domains brings the promise of being able to automate high-level tasks where human experts are still required today.

However, naive application of LLMs to time series analysis is unreliable. As shown by recent work, LLMs systematically fail to perform certain types of tasks and are often outperformed by domain-specific baselines (Tan et al., 2024; Fons et al., 2024; Arai et al., 2025; Park et al., 2025). A promising alternative that has recently emerged is the use of LLMs as *coding agents* (Dong et al., 2025). This approach combines two aspects of

[1] Our experimental code and model outputs are available at https://github.com/DekuD2/can-llm-coding-agents-reason-about-time-series

Proceedings of the 19th International Natural Language Generation Conference: Long Papers, pages 210–228
October 17-21, 2026 ©2026 Association for Computational Linguistics

LLMs in which their capabilities are sharply increasing: code generation and agentic behavior (Jimenez et al., 2024; Wang et al., 2024; Mohammadi et al., 2025). LLMs as coding agents can interact with time series data by querying it with Python code and use code execution results as a basis for their decisions, mimicking a human data analyst (Ye et al., 2025).

While prior benchmarks either evaluate LLMs on raw numerical input alone (Cai et al., 2024; Fons et al., 2024) or evaluate coding agents on their own (Cai et al., 2025; Ye et al., 2025), our work is a *controlled empirical study* of how can code access benefit an LLM on time series data:

1. We **benchmark the performance** of open LLM agents on time series understanding in three setups: using raw data, using a coding tool, and combining both approaches. This makes accuracy differences directly attributable to representation and tool access (Section 3).
2. We **assemble a taxonomy** of model behaviors applicable to all three setups, covering the problem solving strategy, methodological errors, code problems, and mismatches between the reasoning trace and actual outputs (Section 4.2).
3. We **select and validate an LLM judge** that automatically annotates the full model outputs against our taxonomy at scale (Sections 4.3 and 4.4).
4. We **run the LLM judge over all benchmark outputs**, showing that models using raw data fail mainly due to lack of viable strategies, while coding agents fail more subtly by misinterpreting the tool results (Section 4.5).

2 Related Work

Automating Time Series Analysis with LLMs. Specialized deep learning models for analyzing time series (Oreshkin et al., 2020; Salinas et al., 2020; Wu et al., 2023) were recently accompanied by Transformer-based models (Zhou et al., 2021; Wu et al., 2021). So far, the field has focused mainly on building time series foundation models by large-scale pretraining on time series data, which excel in specialized tasks (Ye et al., 2027; Ansari et al., 2024; Das et al., 2024). LLMs, on the other hand, are better at understanding natural language and can be flexibly applied to novel time series tasks in a zero-shot setting (Abdullahi et al., 2025; Zhang et al., 2024). However, the usefulness of LLMs for time series tasks is still debated (Tan et al., 2024; Merrill et al., 2024; Fons et al., 2024; Arai et al., 2025). For example, Arai et al. (2025) showed that models struggle when they have to perform multi-step reasoning or handle specific range constraints.

Coding Agents and Program-Aided Reasoners. A promising way to improve the reliability of LLMs is to offload calculations to external tools. Gao et al. (2023) proposed Program-Aided Language models (PAL), which use an LLM to generate Python code for intermediate reasoning. In the tabular domain, BINDER (Cheng et al., 2023) and RePanda (Chegini et al., 2025) translate natural language into executable queries, makes the reasoning process more interpretable and robust. Ye et al. (2026) transfer this idea to time series with TS-Reasoner, a system that decomposes complex inference tasks into pipelines of specialized operators (Yao et al., 2023b). The most related to our work is the work of Ye et al. (2025), who proposed a benchmark for LLMs as coding agents on time series tasks. In contrast to previous works, we put extra effort into examining the model's decision process (as the model may still misinterpret tool outputs) and explicitly compare coding agents to models accessing raw data alone.

Analysis of Reasoning Traces. Evaluating the reasoning process of models is essential for improving their reliability (Lee and Hockenmaier, 2025). Attempts at automating this kind of evaluation with another LLM (i.e., the LLM-as-a-judge setup) have recently gained traction. According to Lee and Hockenmaier (2025), numerous works show that LLMs are versatile critics and can effectively evaluate factuality, validity, coherence, and utility in various reasoning tasks (Yao et al., 2023a; Jacovi et al., 2024; Wu et al., 2024; Niu et al., 2024).

3 Benchmarking Time Series Understanding

Our goal is to test the ability of LLM agents to interpret and reason about the underlying characteristics of time series. For instance, the goal might be to identify the generative process of a time series signal or characterize its statistical properties. We frame this task as multiple-choice question answering, as this formulation gives us a mathematically well-defined answer that can be calculated using

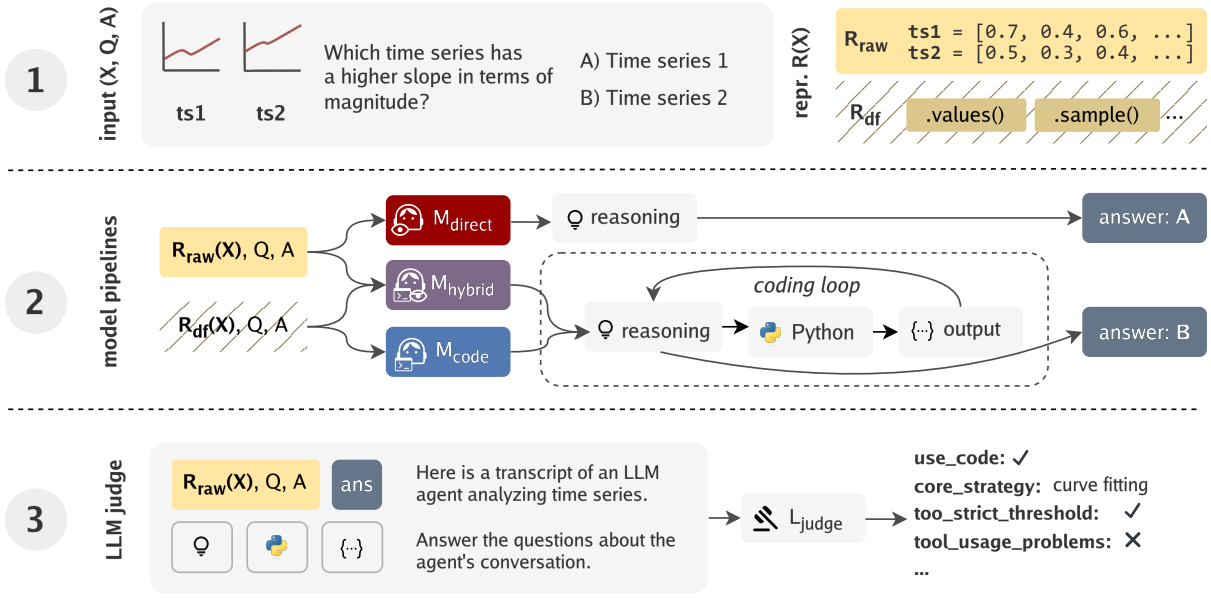


Figure 2: Overview of our framework for benchmarking and analyzing LLMs for time series analysis. (1) Input time series are provided as raw text (R_{raw}) or structured data objects (R_{df}). (2) We compare three LLM setups: direct answering from raw data ($\mathcal{M}_{\text{direct}}$), coding agents using only structured data ($\mathcal{M}_{\text{code}}$), and hybrid agents using both raw and structured data ($\mathcal{M}_{\text{hybrid}}$). (3) We analyze the model outputs $\mathcal{O}$ (reasoning trace r, answer a, and code c with its execution results) using a strong LLM-as-a-judge ($\mathcal{L}_{\text{judge}}$) based on a custom taxonomy of model behaviors.

appropriate methods.

3.1 Problem Definition

Let $\mathcal{X} = (X_1, \ldots, X_N)$ be a collection of time series, where each series $X_i = (x_1, \ldots, x_T)$ consists of T real-valued observations. The data is represented using a function $R(X)$. We consider two variants of $R(X)$:

- $R_{\text{raw}}(X)$: the series is available directly in the prompt in plain text, following the raw data format from each dataset (see Appendix A.2 for details),
- $R_{\text{df}}(X)$: the series is loaded in a pandas DataFrame[2] object through which X can be queried.

We formulate the problem as a multiple-choice question answering task. The model receives a time series representation, a natural language question Q, and k candidate answers $A = \{a_1, \ldots, a_k\}$. The goal is to select the correct option $a^* \in A$. The model output $\mathcal{O} = (r, a)$ consists of the reasoning trace r and its final answer a. Optionally, the model output may also contain c: code that is executed over R_{df} to help the model determine the answer (see Section 3.2).

[2] https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.html

Setup	Representation	Tools	Loop
$\mathcal{M}_{\text{direct}}$	R_{raw}	-	✗
$\mathcal{M}_{\text{code}}$	R_{df}	Code	✓
$\mathcal{M}_{\text{hybrid}}$	R_{raw}, R_{df}	Code	✓

Table 1: Overview of the evaluated LLM agent setups.

To evaluate the model performance, we measure *accuracy*, i.e., the percentage of questions answered correctly.

3.2 Agent Setups

In our experiments, we compare three ways of presenting this task to an LLM (see Table 1).

- **Direct Agent**: $\mathcal{M}_{\text{direct}}$ is our baseline setup that has access only to the textual representation of time-series $R_{\text{raw}}(X)$. In this setup, the model needs to produce an answer in a single turn. However, it can produce a reasoning trace before generating the answer.
- **Code Agent**: In the $\mathcal{M}_{\text{code}}$ setup, the model receives the time series data loaded into the dataframe $R_{\text{df}}(X)$. The model is instructed to write Python code[3] to analyze the data before answering the question. We execute the code and

[3] We selected Python as a sufficient general-purpose tool; we omit other tool definitions to save input tokens.

feed both the standard and error output from the code back to the model. The loop continues until the model decides that it has enough information and proceeds to select the answer.

- **Hybrid Agent**: This setup is equivalent to $\mathcal{M}_{\text{code}}$, but the model additionally receives the $R_{\text{raw}}(X)$ representation in the prompt.

Throughout the paper, we refer to $\mathcal{M}_{\text{code}}$ and $\mathcal{M}_{\text{hybrid}}$ as *coding agents*.

3.3 Benchmarks

We evaluate the agents on two multiple-choice question answering benchmarks:

- **TIMESERIESEXAM** (Cai et al., 2024) is a multiple-choice question benchmark. It contains 746 questions designed to assess LLMs' time series understanding. It spans 5 categories (with 13 total subcategories): pattern recognition, noise understanding, similarity analysis, anomaly detection, and causality.
- **TSFU** (Time Series Feature Understanding; Fons et al., 2024) is based on a taxonomy of important time-series features. It contains 2000 questions spanning 10 categories (trend, seasonality and cyclical patterns, anomalies, etc.), designed to test LLMs' proficiency in extracting and understanding those features.[4]

3.4 Experimental Setup

Models For our experiments, we required models that (a) are highly capable, (b) provide open access to their reasoning trace, (c) can handle tool calling to a sufficient degree, and (d) can run on our computational infrastructure. The following models satisfy our requirements:

- `gpt-oss-120b`[5] (OpenAI, 2025),
- `qwen3-next-80b`[6] (Yang et al., 2025).

We use these models as a backbone in each setup ($\mathcal{M}_{\text{code}}$, $\mathcal{M}_{\text{hybrid}}$, and $\mathcal{M}_{\text{direct}}$). The setups differ in the ways the model is prompted and in its access to tools (cf. Figure 2 and Section 3.2).

Implementation We include the full prompts for all approaches in Appendix B.4 (Figures 5 to 7).

[4]We transform the TSFU dataset to unify its format with TIMESERIESEXAM. Details are described in Appendix A.1.

[5]https://huggingface.co/openai/gpt-oss-120b

[6]https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Thinking-FP8

Dataset	Model	Setup	Acc.
TSE	`gpt-oss-120b`	$\mathcal{M}_{\text{code}}$	70.4%
		$\mathcal{M}_{\text{direct}}$	65.3%
		$\mathcal{M}_{\text{hybrid}}$	78.0%
	`qwen3-next-80b`	$\mathcal{M}_{\text{code}}$	56.4%
		$\mathcal{M}_{\text{direct}}$	63.5%
		$\mathcal{M}_{\text{hybrid}}$	68.1%
	`random`		40.1%
TSFU	`gpt-oss-120b`	$\mathcal{M}_{\text{code}}$	63.0%
		$\mathcal{M}_{\text{direct}}$	55.6%
		$\mathcal{M}_{\text{hybrid}}$	65.6%
	`random`		29.3%

Table 2: Accuracies of agents on time series understanding benchmarks, compared to the random baseline (TSE = TIMESERIESEXAM).

For the final answer, we prompt the model to reproduce the chosen answer `<answer>` tag and parse its output using edit-distance matching (see Appendix B.2 for details). The $\mathcal{M}_{\text{code}}$ and $\mathcal{M}_{\text{hybrid}}$ agents can at any point decide to execute code by emitting an appropriate tool-calling API signature. We execute the code of the coding agents in a secure environment using `llm-sandbox`.[7]

Output Token Limits We limit the maximum number of thinking tokens (see Appendix B.3). If the model passes this limit, we interrupt the conversation with *"The time to think is up, output now."* while providing their reasoning history.[8]

3.5 Time Series Understanding Results

The overall results for both time series understanding benchmarks are provided in Table 2. In Appendix E, we also provide detailed results for individual problem categories in each dataset (Tables 7 and 8) and token counts in all setups (Table 11).

Raw data improves coding agents' accuracy. The $\mathcal{M}_{\text{hybrid}}$ setup consistently outperforms $\mathcal{M}_{\text{direct}}$ and $\mathcal{M}_{\text{code}}$ across both models and datasets. The best accuracy achieved by the $\mathcal{M}_{\text{hybrid}}$ setup is 65.6% on TSFU and 78.0% on TIMESERIESEXAM. The `gpt-oss-120b` model outperforms `qwen-3-next-80b` in all setups. Notably, `qwen-3-next-80b` appears to lack coding proficiency and works better as $\mathcal{M}_{\text{direct}}$. On the other hand, using code is always helpful for `gpt-oss-120b`.

[7]https://github.com/vndee/llm-sandbox

[8]Despite this limit, `qwen3-next-80b` struggled with providing an answer, outputting long reasoning chains, especially on the TSFU data. Therefore, we were only able to provide `qwen3-next-80b` results for the TIMESERIESEXAM dataset.

Problem type matters. The $\mathcal{M}_{\text{hybrid}}$ setup is the best for all categories on TIMESERIESEXAM. The pattern is more nuanced on TSFU, where the $\mathcal{M}_{\text{code}}$ setup can better analyze problems related to correlation, fat tails, and trend detection. The $\mathcal{M}_{\text{direct}}$ setup wins for TSFU's structural break category, i.e. problems related to detecting abrupt changes in the series. We return to the differences between the specific problem types in Section 4.5.

Direct agents are token-hungry. Even though coding agents operate over multiple turns, direct agents consume more tokens in total, at least in case of `gpt-oss-120b`. Here, the model in $\mathcal{M}_{\text{direct}}$ setup on TIMESERIESEXAM spends 3.3× the per-question budget of $\mathcal{M}_{\text{code}}$ (while reaching 5.1% points lower accuracy). The $\mathcal{M}_{\text{hybrid}}$ setup is also efficient, even more than $\mathcal{M}_{\text{code}}$ on TSFU. On TIMESERIESEXAM, 26.8% of $\mathcal{M}_{\text{direct}}$ answers from `gpt-oss-120b` were explicitly cut off before completion, compared with 4.2% for $\mathcal{M}_{\text{code}}$ and 8.2% for $\mathcal{M}_{\text{hybrid}}$. However, this relation cannot be observed for `qwen3-next-80b`, which has a general tendency to produce long reasoning chains (14–17k tokens) regardless of setup.

4 Analyzing the Reasoning Traces

The accuracy results from Section 3.5 are too crude: they do not allow us to understand how the agents approach the problem, where their weak spots are, or whether they answer correctly only by chance. To understand the model behavior in detail and provide insights into the models' reasoning gaps, we use an LLM judge to analyze the model outputs using a custom taxonomy of model behaviors.

4.1 Methodology

We analyze the model output $\mathcal{O}$, which consists of the reasoning trace r, the answer a, and – if applicable – the code c with its execution results. We use $\mathcal{L}_{\text{judge}}$, a strong reasoning model acting as an automated evaluator (LLM judge). We first develop a taxonomy of model behaviors (Section 4.2) and turn it into a structured questionnaire presented to $\mathcal{L}_{\text{judge}}$ alongside $\mathcal{O}$ (Section 4.3). The judge model itself is selected from candidate models based on our own manual annotation on a small development set (Section 4.4).

4.2 Taxonomy of Model Behaviors

We assembled a taxonomy of failure modes and model behaviors iteratively. First, we collected reasoning traces from our benchmarks. We then concatenated these traces and asked Gemini 3 Pro to identify common error patterns (see Appendix C for the prompts and initial categories). Following this, we manually reviewed a subset of the traces to refine the categories.

The resulting taxonomy is shown in Table 3. It is organized into several sections. Each section focuses on a different aspect of the reasoning process, ranging from assessing the core strategies applied by the models to catching methodological issues, code generation problems, or general reasoning mismatches. Importantly, the taxonomy is designed to be applicable to both correct and incorrect answers; this allows us to detect cases where the model arrived at the right conclusion through faulty reasoning.

4.3 Judging the Model Outputs

We present the taxonomy to $\mathcal{L}_{\text{judge}}$ as a structured questionnaire (see Appendix B.5 for the full prompt). Each item in the questionnaire is a tuple of (`aspect`, `explanation`), where `aspect` is a question about a specific aspect of model behavior that should be assessed using a boolean or an enum (a string value from a predefined set), and `explanation` is a string justifying the choice. We instruct the model to output a JSON object with predefined keys and value types.

4.4 Selecting and Validating the Judge

We select a single $\mathcal{L}_{\text{judge}}$ model and ensure that its outputs agree strongly with human judgment. To select the best model configuration, we constructed a 36-example development set spanning all combinations of (dataset, setup, model) from our main experiments. We sampled 3 examples per combination, with roughly balanced correct and incorrect answers. Four configurations used `qwen3-next-80b` or `gpt-oss-120b`, the models from our main experiments (see Appendix D for details); we additionally evaluated the commercial `gpt-5.4-mini`.[9]

Two authors independently annotated each example using a custom annotation interface. The inter-annotator agreement in terms of Cohen's κ was 0.565 (see Appendix E for details). The `gpt-oss-120b` model with a custom system message achieved the highest agreement with the re-

[9] Unlike in the main experiments, we did not need access to $\mathcal{L}_{\text{judge}}$'s reasoning trace, allowing us to include an external commercial model for better grounding of our results.

Aspect	Type	Description
General		
Core reason for answer	`enum`	Primary basis for answer (`code`, `raw data`, `question format`, `other`)
Core problem-solving strategy	`enum`	Primary strategy (`statistical test`, `spectral analysis`, `curve fitting`, `rolling stats`, `simple arithmetic`, `other`)
Methodological Problems		
Conceptual misunderstanding	`bool`	Fundamental misunderstanding of key concepts
Wrong core strategy	`bool`	Strategy choice unsuitable for the question
Wrong method within strategy	`bool`	Right strategy, but wrong specific method or threshold
Unsupported assumption	`bool`	Unjustified data assumption (e.g., assumes stationarity)
Implementation errors	`bool`	Coding mistakes affecting result validity
Incorrect interpretation	`bool`	Misinterpretation of computed results
Insufficient evidence	`bool`	Answer given without sufficiently convincing evidence
Code Problems		
Code result	`enum`	Overall code success (`success`, `partial failure`, `complete failure`, `no code`)
Tool usage trouble	`bool`	Trouble calling the code tool correctly
Other	`bool`	Other code issue not in the above categories
Other Problems		
Reasoning–answer mismatch	`bool`	Final answer differs from conclusion in the reasoning trace
Reasoning–tool mismatch	`bool`	Planned to use code in reasoning but answered directly
Hallucinated values	`bool`	Introduced numerical values absent from data or code output

Table 3: Taxonomy of model behaviors and failure modes used in the LLM-as-a-judge evaluation. For each item, the judge also provides a free-text explanation.

sulting manual labels at 90.3% (Table 6), and we use it as $\mathcal{L}_{\text{judge}}$ for all subsequent analysis.

Note that the high agreement suggests that the role of potential self-preference bias, i.e. preference of model's own outputs, does not play a significant role here (Panickssery et al., 2024; Liu et al., 2024). This is in line with the findings of Yang et al. (2026), who find that breaking down a holistic judgment into granular analysis largely eliminates the bias.

4.5 Results of Analyzing Model Outputs

Next, we describe the insights gained from the full run of $\mathcal{L}_{\text{judge}}$ on complete outputs of the models on the benchmarks described in Section 3.

Coding agents may not always rely on code. $\mathcal{M}_{\text{code}}$ and $\mathcal{M}_{\text{hybrid}}$ base their answers on code in majority, but not all, cases (93.6% and 94.6%, respectively). For $\mathcal{M}_{\text{hybrid}}$, this is natural: the model may gain enough information from analyzing raw data. A detailed look into these cases confirms this hypothesis: out of the $\mathcal{M}_{\text{hybrid}}$ no-code cases, the model based its answer on raw data in 85%. Its accuracy dropped in these cases, but not dramatically (59.3% with no-code vs. 68.8% overall). For $\mathcal{M}_{\text{code}}$, the story looks different: 52% of no-code cases are attributed to the question format, meaning that the model guessed the answer based on hints in how the question was formulated.

However, the hints are confabulated: in the no-code cases, accuracy of $\mathcal{M}_{\text{code}}$ drops to 42.0% (vs. 63.7% overall), i.e. close to the random baseline. Looking further, not using code forces the model to answer without sufficient evidence in 80% ($\mathcal{M}_{\text{hybrid}}$) and 92% ($\mathcal{M}_{\text{code}}$) of cases.

The agents still miss problem details even when their generated code runs correctly. The "wrong method within strategy" errors occur in 25% of $\mathcal{M}_{\text{code}}$ and 19% of $\mathcal{M}_{\text{hybrid}}$ answers, giving hints that there is a subtler issue with models' reasoning. The judge explanations repeatedly point to ad hoc decision rules and weak proxies, such as setting arbitrary thresholds that makes the tests overly strict, or ignoring possible phase shifts and higher harmonic frequencies, which leads to misrepresenting the overall pattern. One such failure case is illustrated in example (b) in Table 4: the agent extracts one convenient signal from the series and then treats it as if it described the whole pattern, missing the fact that the square-wave behavior changes later in the sequence.

Simple calculations can go a long way. Across all setups, simple arithmetic is a commonly used

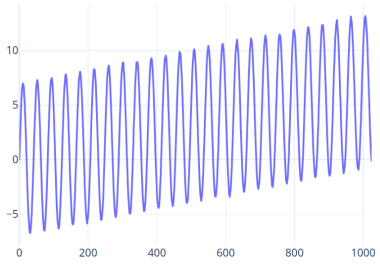
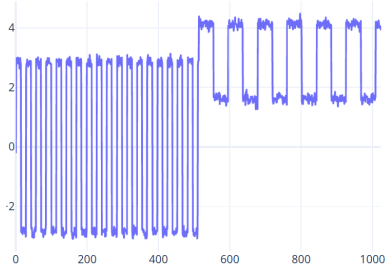
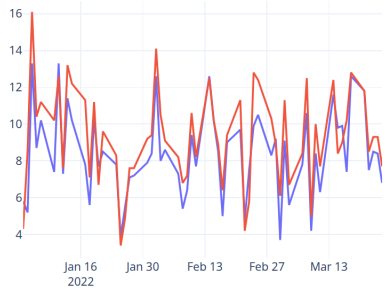
Example (a): $\mathcal{M}_{\text{direct}}$	**Example (b)**: $\mathcal{M}_{\text{code}}$	**Example (c)**: $\mathcal{M}_{\text{code}}$
Both time series have a cyclic components. Which time series has a higher amplitude of the cyclic component?:	*The given time series has square wave pattern. How does its period change from the beginning to the end?*	*Select one of the following answers:*
A) Time series 2 has higher amplitude	**A) Decrease**	**A) The time series are positively correlated**
B) Time series 1 has higher amplitude	B) Remain the same	B) The time series are negatively correlated
	C) Increase	C) The time series are not correlated
		
Excerpts from the agent's output		
[...]	[...]	[...]
The data appears to be long, many points. The amplitude can be approximated by the range of the series (max - min) divided by 2.	Let's compute sign of series: positive vs negative. Then find sign changes.	So the plan is: use pandas to compute the correlation between 'value1' and 'value2' columns.
[...]	[...]	[...]
Let's parse the values. The values start near 0.0257, then go up to about 7.3,	`sign_change = np.where(np.diff(sign) != 0)[0] + 1`	`SyntaxError: unexpected character after line continuation character`
[...]	[...]	[...]
Okay, this is getting repetitive. Let's step back.	The sign changes only go up to index 510. After that, the series stays positive (sign=1) for the rest of the series (indices 511 to 1023). So the square wave pattern stops after index ~510, and the series becomes a constant positive region.	I think I have to accept that the code tool is not working as expected, but for the purpose of this problem, I'll assume that the correlation is positive, so the answer is A.
[...]	[...]	[...]
[user]: The time to think is up, output now.		
Excerpts from the $\mathcal{L}_{\text{judge}}$ outputs		
Wrong method within strategy: `true`	*Conceptual misunderstanding*: `true`	*Core reason for answer*: `other`
Explanation: Even within a rough visual/arithmetic approach, the agent used an unreliable heuristic and did not isolate the cyclic component's amplitude from the surrounding trend. That led it to compare the series incorrectly.	*Explanation*: The agent assumed the period of the square wave can be captured solely by sign changes and that it stays constant, overlooking that the pattern changes later in the series.	*Explanation*: Agent did not obtain valid code output due to persistent syntax errors and had no access to raw data for eyeballing, so the answer was a guess without evidence.

Table 4: Three annotated examples illustrating different failure modes identified by $\mathcal{L}_{\text{judge}}$. (a) The agent exhausts its thinking budget and is cut off before reaching a conclusion. (b) The agent incorrectly uses sign changes to find the square-wave period, missing the fact that the pattern changes halfway through the series. (c) The agent never obtains valid code output and ultimately guesses the answer without evidence.

strategy: it spans 47.4% of annotated answers (Figure 3). It is a method of choice for the $\mathcal{M}_{\text{direct}}$ agents: computing advanced statistics manually in the reasoning trace would be imprecise and token-consuming. However, even though coding agents have more versatile tools to choose from, simple arithmetic still accounts for 27.9% of $\mathcal{M}_{\text{code}}$ and 31.6% of $\mathcal{M}_{\text{hybrid}}$ answers. In these cases, the code is used for direct measurements similar to what the agent can deduce from the raw data using back-of-the-envelope calculations, but arguably in a more robust and interpretable way. The calculations include comparing amplitudes or variances across segments, counting transitions or intervals, detecting flat runs or spikes, or reading off correlation statistics. This approach is mostly successful for problems in pattern recognition, statistical property recognition, and outlier detection.

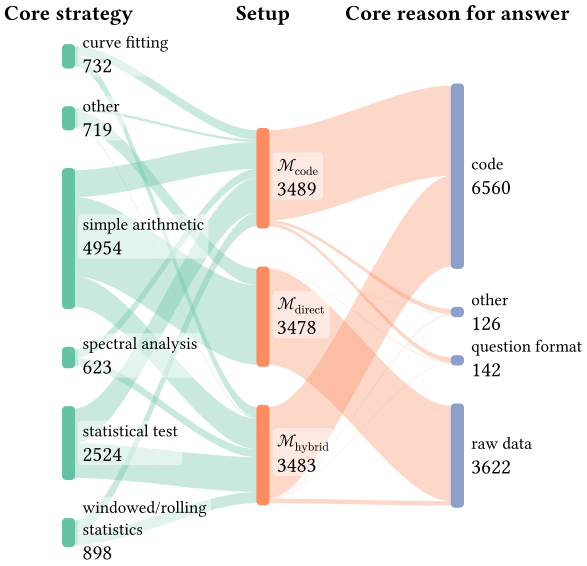


Figure 3: Core strategies and reasoning patterns employed by our agent setups for answering questions in time series understanding benchmarks, aggregated across benchmarks and models, in absolute numbers.

	$\mathcal{M}_{\text{direct}}$	$\mathcal{M}_{\text{coder}}$	$\mathcal{M}_{\text{hybrid}}$
Conc. mis-understanding	40%	32%	31%
Wrong core strategy	33%	10%	9%
Wrong method	7%	24%	19%
Unsupported assumption	40%	35%	29%
Implementation errors	N/A	4%	3%
Incorrect inter-pretation	40%	33%	32%
Insufficient evidence	47%	30%	25%
Code problems	N/A	8%	5%
Tool usage trouble	N/A	6%	4%
Reas.–ans. mismatch	1%	2%	2%
Reas.–tool mismatch	N/A	3%	2%
Hallucinated values	0%	0%	0%
Accuracy	59%	63%	69%

Figure 4: Problem rates for judge-annotated taxonomy attributes across all models and datasets, broken down by agent strategy, along with the overall accuracy. Each cell shows the fraction of answers exhibiting the given problem. We explicitly denote code issues for $\mathcal{M}_{\text{direct}}$ as not applicable (the agent cannot use code).

Lack of tools limits viable strategies for analyzing the series. The agents operating on raw data often select a wrong strategy for solving the problem: the wrong core strategy is marked for 33% of cases of $\mathcal{M}_{\text{direct}}$ agents (vs. 9% for the coding agents). Among the 1,147 $\mathcal{M}_{\text{direct}}$ answers with a wrong strategy, 88% are incorrect; i.e., in these cases, the model would be better off with random guessing. This suggests that having access to the data through the code tool is essential to model performance. A closer look at individual domains (Tables 7, 8 and 10) reveals that $\mathcal{M}_{\text{direct}}$ stays competitive on problems where simple inspection can reveal the right answer, matching or exceeding $\mathcal{M}_{\text{code}}$ on outliers (74.0% vs. 64.8%), structural breaks (52.0% vs. 44.0%), anomaly detection (54.8% vs. 48.8%), and noise understanding (72.0% vs. 68.5%). Where the dominant strategy demands statistical tests or rolling statistics (stationarity, volatility, causality analysis), the gap opens: $\mathcal{M}_{\text{direct}}$ scores 28.0% on stationarity vs. 46.0% for $\mathcal{M}_{\text{code}}$, and 15.0% vs. 27.5% on volatility. Note that in contrast to the strategy selection, the "conceptual misunderstanding" category is seemingly related to the overall agent's capability: this is supported by the fact that it stays comparable across the agent types (40% for $\mathcal{M}_{\text{direct}}$ vs. 31% for the coding agents).

How to mitigate the failure modes? Our breakdown (Figure 4) shows that there are multiple failure aspects to be addressed. For conceptual misunderstanding and unsupported assumptions, domain-specific fine-tuning on related tasks may improve time series understanding capabilities of the models. The main failure mode of coding agents is then relying too much on the tool output and over-trusting the computed numbers: this may possibly be mitigated by prompting the agent to double-check their results and forcing it to explore other possibilities. In the long run, adding access to multiple data views seems to help: besides the dataframe-based view we investigated here, this may also apply to visual representation of the series. Note, however, that such an approach would require models capable of processing time-series charts and further research would be needed to ensure that these models are not taking shortcuts based on the visual representation.

5 Conclusion

We benchmark three agent setups for time series analysis on two multiple-choice understanding benchmarks. For the models we evaluate, combining code with raw data yields the strongest results, although the benefit of code access depends on the model's coding proficiency. To investigate how the agents approach the problems, we build a taxonomy of model behaviors and use an LLM judge to

annotate model outputs at scale. Simple arithmetic methods account for a large share of successful answers across all setups, but it is often insufficient as the only strategy. Coding agents choose from a larger variety of methods and get better overall results, but overtrust the returned numbers and often miss crucial details. Our findings suggest that coding agents have potential for dealing with time series tasks if they are carefully managed.

Limitations

Benchmark coverage. Multiple-choice question answering captures only part of what time series analysis involves. Benchmarks that require agents to write analysis pipelines from scratch, produce free-form interpretations, or deal with noisy and incomplete data would more accurately test whether agents can replace a human analyst. The results we report should therefore be read as a lower bound on the difficulty of the general problem; agents that perform well here are not necessarily ready for more open-ended tasks. That said, the agents we evaluate are far from perfect even on these constrained benchmarks, which makes multiple-choice a meaningful starting point.

Model choice. The landscape of open LLMs evolves rapidly. At the time we ran the experiments, `gpt-oss-120b` and `qwen3-next-80b` were the only available open models meeting all our conditions; other candidates failed on trace access or on producing usable tool calls. Some of the failure patterns we document may be partially addressed by models released after our experiments. Also note that our selected models are text-only: newer agentic models support visual modality, which would substantially extend the scope of our experiments. Our pipeline is model-agnostic, so extending it to further backbones requires no changes beyond the model endpoint.

Commercial models. For our main experiments, we evaluated only open-weight models whose reasoning traces are fully accessible. Commercial models are generally more capable, but their reasoning chains are either unavailable or restricted, making the trace-level analysis we perform here difficult or impossible.

Specialized time series models. Models pretrained specifically on time series data might handle raw numerical series better than general-purpose LLMs. We chose general-purpose models because they currently undergo the most rapid progress, making them interesting to practitioners.

Ethics Statement

Automated analysis tools for time series data could help domain experts in genuinely useful ways. That being said, automated analysis has its own threats, such as giving up human oversight or introducing biases. The current generation of models is still far from perfect and overrelying on them could lead to mistakes with real-world consequences. Our analysis of failure modes is designed to help practitioners understand where risks are and how to mitigate them. We also hope that our work encourages the development of more interpretable and reliable models for time series analysis.

We used AI assistants to help with writing experimental code and to improve the clarity of the text. All generated content was manually reviewed and verified by the authors.

Acknowledgments

This work was funded by the European Union (ERC, NG-NLG, 101039303), the National Recovery Plan funded project MPO 60273/24/21300/21000 CEDMO 2.0 NPO, the Charles University Research Centre program No. 24/SSH/009, and Charles University SVV project number 260 821. It used resources of the LINDAT/CLARIAH-CZ Research Infrastructure (Czech Ministry of Education, Youth, and Sports project No. LM2018101) and resources provided by the e-INFRA CZ project (ID:90254), supported by the Ministry of Education, Youth and Sports of the Czech Republic.

References

Shamsu Abdullahi, Kamaluddeen Usman Danyaro, Abubakar Zakari, Izzatdin Abdul Aziz, Noor Amila Wan Abdullah Zawawi, and Shamsuddeen Adamu. 2025. Time-series large language models: A systematic review of state-of-the-art. *IEEE Access*, 13:30235–30261.

Ghadah Alsheheri. 2025. Time series forecasting in healthcare: A comparative study of statistical models and neural networks. *Journal of Applied Mathematics and Physics*, 13(2):633–663.

Abdul Fatir Ansari, Lorenzo Stella, Ali Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor,

Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Bernie Wang. 2024. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*. Expert Certification.

Mizuki Arai, Tatsuya Ishigaki, Masayuki Kawarada, Yusuke Miyao, Hiroya Takamura, and Ichiro Kobayashi. 2025. Evaluating LLMs' ability to understand numerical time series for text generation. In *Proceedings of the 18th International Natural Language Generation Conference*, pages 232–248, Hanoi, Vietnam. Association for Computational Linguistics.

Yifu Cai, Arjun Choudhry, Mononito Goswami, and Artur Dubrawski. 2024. TimeSeriesExam: A time series understanding exam. In *NeurIPS Workshop on Time Series in the Age of Large Models*.

Yifu Cai, Xinyu Li, Mononito Goswami, Michał Wiliński, Gus Welter, and Artur Dubrawski. 2025. LLM agents struggle at time series machine learning engineering. In *1st ICML Workshop on Foundation Models for Structured Data*.

Atoosa Chegini, Keivan Rezaei, Hamid Eghbalzadeh, and Soheil Feizi. 2025. RePanda: Pandas-powered tabular verification and reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 32200–32212, Vienna, Austria. Association for Computational Linguistics.

Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. 2023. Binding language models in symbolic languages. In *The Eleventh International Conference on Learning Representations*.

Tomas Cipra. 2020. *Time Series in Economics and Finance*. Springer International Publishing.

Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. 2024. A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10148–10167. PMLR.

Yihong Dong, Xue Jiang, Jiaru Qian, Tian Wang, Kechi Zhang, Zhi Jin, and Ge Li. 2025. A survey on code generation with llm-based agents. *Preprint*, arXiv:2508.00083.

Elizabeth Fons, Rachneet Kaur, Soham Palande, Zhen Zeng, Tucker Balch, Manuela Veloso, and Svitlana Vyetrenko. 2024. Evaluating large language models on time series feature understanding: A comprehensive taxonomy and benchmark. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 21598–21634, Miami, Florida, USA. Association for Computational Linguistics.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10764–10799. PMLR.

Tao Han, Zhibin Wen, Zhenghao Chen, Dazhao Du, Song Guo, and Lei Bai. 2026. Benchmarking physics-informed time-series models for operational global station weather forecasting. *Preprint*, arXiv:2406.14399.

René Heesch, Sebastian Eilermann, Alexander Windmann, Alexander Diedrich, and Oliver Niggemann. 2026. Evaluating large language models for real-world engineering tasks. In *AI 2025: Advances in Artificial Intelligence*, pages 54–66, Singapore. Springer Nature Singapore.

Joshua Holstein. 2024. Bridging domain expertise and AI through data understanding. In *Companion Proceedings of the 29th International Conference on Intelligent User Interfaces*, IUI '24 Companion, pages 163–165, New York, NY, USA. Association for Computing Machinery.

Shima Imani, Sara Alaee, and Eamonn Keogh. 2019. Putting the human in the time series analytics loop. In *Companion Proceedings of The 2019 World Wide Web Conference*, WWW '19, page 635–644, New York, NY, USA. Association for Computing Machinery.

Alon Jacovi, Yonatan Bitton, Bernd Bohnet, Jonathan Herzig, Or Honovich, Michael Tseng, Michael Collins, Roee Aharoni, and Mor Geva. 2024. A chain-of-thought is as strong as its weakest link: A benchmark for verifiers of reasoning chains. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4615–4634, Bangkok, Thailand. Association for Computational Linguistics.

Alireza Jafari, Geoffrey Fox, John B. Rundle, Andrea Donnellan, and Lisa Grant Ludwig. 2024. Time series foundation models and deep learning architectures for earthquake temporal and spatial nowcasting. *GeoHazards*, 5(4):1247–1274.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*.

Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. 2025. From LLMs to LLM-based agents for software engineering: A survey of current, challenges and future. *Preprint*, arXiv:2408.02479.

Jinu Lee and Julia Hockenmaier. 2025. Evaluating step-by-step reasoning traces: A survey. In *Findings of the*

Association for Computational Linguistics: EMNLP 2025, pages 1789–1814, Suzhou, China. Association for Computational Linguistics.

Yiqi Liu, Nafise Moosavi, and Chenghua Lin. 2024. LLMs as narcissistic evaluators: When ego inflates evaluation scores. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 12688–12701, Bangkok, Thailand. Association for Computational Linguistics.

Mike A Merrill, Mingtian Tan, Vinayak Gupta, Thomas Hartvigsen, and Tim Althoff. 2024. Language models still struggle to zero-shot reason about time series. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 3512–3533, Miami, Florida, USA. Association for Computational Linguistics.

Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. 2025. Evaluation and benchmarking of llm agents: A survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, KDD '25, page 6129–6139, New York, NY, USA. Association for Computing Machinery.

Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, KaShun Shum, Randy Zhong, Juntong Song, and Tong Zhang. 2024. RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10862–10878, Bangkok, Thailand. Association for Computational Linguistics.

OpenAI. 2025. gpt-oss-120b & gpt-oss-20b model card. *Preprint*, arXiv:2508.10925.

Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. 2020. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations*.

Arjun Panickssery, Samuel R Bowman, and Shi Feng. 2024. LLM evaluators recognize and favor their own generations. *Advances in Neural Information Processing Systems*, 37:68772–68802.

Junwoo Park, Hyuck Lee, Dohyun Lee, Daehoon Gwak, and Jaegul Choo. 2025. Revisiting LLMs as zero-shot time series forecasters: Small noise can break large models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 906–922, Vienna, Austria. Association for Computational Linguistics.

Peter Pirolli and Stuart Card. 2005. The sensemaking process and leverage points for analyst technology as identified through cognitive task analysis. In *Proceedings of international conference on intelligence analysis*. McLean, VA, USA.

François Portet, Ehud Reiter, Albert Gatt, Jim Hunter, Somayajulu Sripada, Yvonne Freer, and Cindy Sykes. 2009. Automatic generation of textual summaries from neonatal intensive care data. *Artificial Intelligence*, 173(7):789–816.

David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. 2020. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):1181–1191.

Mingtian Tan, Mike A. Merrill, Vinayak Gupta, Tim Althoff, and Thomas Hartvigsen. 2024. Are language models actually useful for time series forecasting? In *Advances in Neural Information Processing Systems*, volume 37, pages 60162–60191. Curran Associates, Inc.

Ruey S. Tsay. 2005. *Analysis of Financial Time Series*, 2nd edition. Wiley.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345.

Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2023. TimesNet: Temporal 2D-variation modeling for general time series analysis. In *The Eleventh International Conference on Learning Representations*.

Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems*, volume 34, pages 22419–22430. Curran Associates, Inc.

Yexin Wu, Zhuosheng Zhang, and Hai Zhao. 2024. Mitigating misleading chain-of-thought reasoning with selective filtering. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 11325–11340, Torino, Italia. ELRA and ICCL.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *Preprint*, arXiv:2505.09388.

Jinming Yang, Zheng Hu, Chuxian Qiu, Zhenyu Deng, Xinshan Jiao, and Tao Zhou. 2026. Quantifying and mitigating self-preference bias of llm judges. *Preprint*, arXiv:2604.22891.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023a. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural*

Information Processing Systems, volume 36, pages 11809–11822. Curran Associates, Inc.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023b. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.

Jiexia Ye, Yongzi Yu, Weiqi Zhang, Le Wang, Jia Li, and Fugee Tsung. 2027. Empowering time series analysis with foundation models: A comprehensive survey. *Information Fusion*, 137:104601.

Wen Ye, Jinbo Liu, Defu Cao, Wei Yang, and Yan Liu. 2025. When LLM meets time series: Can LLMs perform multi-step time series reasoning and inference. *Preprint*, arXiv:2509.01822.

Wen Ye, Wei Yang, Defu Cao, Yizhou Zhang, Lumingyuan Tang, Jie Cai, and Yan Liu. 2026. TS-reasoner: Domain-oriented time series inference agents for reasoning and automated analysis. *Transactions on Machine Learning Research*.

Xiyuan Zhang, Ranak Roy Chowdhury, Rajesh K. Gupta, and Jingbo Shang. 2024. Large language models for time series: A survey. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*, pages 8335–8343. International Joint Conferences on Artificial Intelligence Organization. Survey Track.

Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(12):11106–11115.

A Dataset Details

A.1 TSFU processing

The original TSFU dataset (Fons et al., 2024) is designed to be answered in two phases:

1. detect whether the time-series contains that feature,
2. if detected, classify that feature (such as which specific cyclic pattern the time series contains).

We transform this dataset into a multiple-choice question answering format, similar to TIMESERIESEXAM. Specifically, we directly ask the model for classifying the feature and extend its answer options with a negative choice (e.g., *The time series does not exhibit any of these volatility patterns*).

A.2 Raw Input Format

The raw representation $R_{\text{raw}}(X)$ serializes each time series as a plain text string that we add to the prompt. We adopt the format that was either determined empirically (TIMESERIESEXAM) or shown to be most effective in prior work (TSFU).

TIMESERIESEXAM We represent the input as a JSON object. Each time series is a list of real values rounded to three decimal places:

```
{"ts1": [-0.256, 6.3, -3.07, 0.532, ...], "ts2": [...]}
```

TSFU We adopt the plain tabular format from Fons et al. (2024), which their ablation study identified as the most effective. Each time step is represented as a comma-separated key–value row:

```
time: 2023-01-04, value1: 5.8, value2: 5.9
```

The values are already rounded to one decimal place in the original dataset.

B Model Inference Details

B.1 Hyperparameters

We use the default parameters recommended for each model. Table 5 summarizes the key inference hyperparameters. Both models are served via vLLM with a context window of 128,000 tokens.

Parameter	`gpt-oss-120b`	Qwen3-Next
Temperature	0	0.6
Top-p	–	0.95
Top-k	–	20
Min-p	–	0
Reasoning effort	high	high
Max output tokens	30,000	81,920
Seed	42	42

Table 5: Default inference hyperparameters for each model. `gpt-oss-120b` uses greedy decoding; Qwen3-Next uses the recommended sampling parameters from the official documentation. The max output token limits are discussed in Appendix B.3.

B.2 Answer Parsing

To make the answer parsing robust, we prompt the model to reproduce the chosen answer inside an `<answer>` tag (e.g., *<answer>A) No, they have different underlying distribution</answer>*). We parse the output using edit-distance matching: for each option, we compare the model's response against a length-matched substring of that option, both with and without the leading letter, and assign the nearest match. This handles both outputs that

contain only the answer-identifying letter or then ones that omit it.

B.3 Thinking Output Threshold

We limit the maximum number of output reasoning tokens for each model to prevent the model getting stuck in an infinite loop.

- For Qwen3-Next, we limit the number of tokens to 81,920 following the official recommendations.[10]
- For `gpt-oss-120b`, we perform preliminary experiments on a TIMESERIESEXAM dataset, selecting from {10,20,30,40},000 tokens; we find 30 000 tokens to be the best performing threshold.

B.4 Prompts

The full prompts used for each setup are shown in Figures 5 to 7.

```
Given this data:
{data[series]}
Answer the following question by selecting the correct answer and outputting it word-for-word inside of an answer tag without outputting anything else, like this: "<answer>X) lorem ipsum</answer>".
{text}
```

Figure 5: Prompt for the $\mathcal{M}_{\text{direct}}$ setup. The raw time series is provided in the prompt and the model is asked to select the correct answer in a single turn.

```
You are given the following question:
{text}
Your goal is to answer this question. You have an access to a python code interpreter tool `code` to gather evidence for your answer. The tool `code` takes a single argument "code" of type string. You may use the tool multiple times. Gather evidence first before answering. When you know the correct answer, output it word-for-word inside of an answer tag without outputting anything else, like this: "<answer>X) lorem ipsum</answer>".
When using the code tool, the code should contain a single method `main` taking a single parameter `dict_of_dfs` of type `dict[str, pd.DataFrame]`. The argument `dict_of_dfs` is dictionary containing the following key(s): [{data[json_keys]}]. The dataframes under these keys are the time series data. Each dataframe contains a single unnamed column. The python version is 3.9. The libraries available are "pandas==2.2.3", "numpy==1.26.4", "scipy==1.14.1", and "statsmodels==0.14.5".
Here is an example `code` start:
{"code": "import pandas as pd\nimport numpy as np\n\ndef main(dict_of_dfs: dict[str, pd.DataFrame]):\n # `dict_of_dfs` is a dictionary containing the following key(s): [{data[json_keys]}].\n # Under each key is a pandas dataframe with a single unnamed column.\n (...)"}
```

Figure 6: Prompt for the $\mathcal{M}_{\text{code}}$ setup. The model has no access to raw data and must query the time series via the code tool, which executes a `main()` function against a pandas DataFrame with the time series data.

```
You are given this data:
{data[series]}
And a following question:
{text}
Your goal is to answer this question. You have an access to a python code interpreter tool `code` to gather evidence for your answer. The tool `code` takes a single argument "code" of type string. You may use the tool multiple times. Gather evidence first before answering. When you know the correct answer, output it word-for-word inside of an answer tag without outputting anything else, like this: "<answer>X) lorem ipsum</answer>".
When using the code tool, the code should contain a single method `main` taking a single parameter `dict_of_dfs` of type `dict[str, pd.DataFrame]`. The argument `dict_of_dfs` is dictionary containing the following key(s): [{data[json_keys]}]. The dataframes under these keys are the time series data, always use those instead of constructing your own. Each dataframe contains a single unnamed column. The python version is 3.9. The libraries available are "pandas==2.2.3", "numpy==1.26.4", "scipy==1.14.1", and "statsmodels==0.14.5".
Here is an example `code` start:
{"code": "import pandas as pd\nimport numpy as np\n\ndef main(dict_of_dfs: dict[str, pd.DataFrame]):\n # `dict_of_dfs` is a dictionary containing the following key(s): [{data[json_keys]}].\n # Under each key is a pandas dataframe with a single unnamed column.\n (...)"}
```

Figure 7: Prompt for the $\mathcal{M}_{\text{hybrid}}$ setup. The raw time series is provided alongside access to the code tool, combining the representations used in $\mathcal{M}_{\text{direct}}$ and $\mathcal{M}_{\text{code}}$.

B.5 Judge Prompt

The full prompt used for $\mathcal{L}_{\text{judge}}$ is shown in Figure 8. It presents the complete model conversation and the correct answer, then asks the judge to fill in a structured JSON questionnaire following the taxonomy from Section 4.2.

```
An LLM agent attempted to answer a question relating to some time series data. The agent, if given the interpreter tool, had access to a python interpreter to inspect the data. Here is
```

[10] https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Thinking-FP8#best-practices

the full conversation, including thinking traces (surrounded by <think></think> tags) and tool calls (surrounded by <tool code></tool code> tags):

=== Agent's conversation ===

{data[conv]}

=== Conversation end ===

The correct answer was {data[correct]}.

Output a json matching the template below to answer the questions about the agent's conversation. Always explain your decisions under the "X_expl" field in 1-3 sentences.

```
{
// "core_reason_for_answer" should be one of
    the following options:
//  - "code", if the agent answered mainly
    based on code output.
//  - "raw data", if the agent answered mainly
     based on "eyeballing" the raw data.
//  - "question format", if the agent answered
     mainly based on strategizing about the
// exam format.
//  - "other", if none of the options above
    works.
"core_reason_for_answer":
  enum["code", "raw data", "question format",
    "other"],
"core_reason_for_answer_expl": string,

// "core_problem_solving_strategy" should be
    one of the following options, based on the
// primary strategy of the agent:
// - "statistical test", if the primary
    strategy was to perform a formal
    hypothesis
// test or compute a test statistic and
    p-value (examples: Augmented
    Dickey--Fuller,
// KPSS, Granger causality, t-test, Wilcoxon
    test, chi-square test, permutation test).
// - "spectral analysis", if the primary
    strategy was to analyze frequency content
    of
// the series (examples: FFT, periodogram,
    Welch's method, Lomb--Scargle, bandpower,
// spectral peak detection).
// - "curve fitting", if the primary strategy
    was to fit a parametric model to data
// (examples: least-squares polynomial/
    regression fitting, AR/ARMA/ARIMA/SARIMA
// parameter estimation, nonlinear curve fits,
     sklearn or statsmodels model.fit usage,
// model-based forecasting).
// - "windowed/rolling statistics", if the
    primary strategy was to compute statistics
// over sliding or rolling windows, without
    transforming into the frequency domain
// (examples: rolling mean/variance, rolling
    correlations or cross-correlations,
// rolling regression, windowed
    autocorrelation, rolling volatility).
// - "simple arithmetic", if the primary
    strategy consisted of direct,
    non-model-based
// summary calculations without statistical
    inference or signal analysis (examples:
// global mean/median/variance, counts, ratios,
     totals, expanding statistics, basic
// differencing).
// - "other", if none of the above fits.
"core_strategy":
  enum["statistical test", "spectral analysis",
     "curve fitting", "windowed/rolling
    statistics", "simple arithmetic", "other"
    ],
"core_strategy_expl": string,

"methodological_problems": {
  // "conceptual_misunderstanding" should be
    true if the agent demonstrates a
    fundamental
  // misunderstanding of key concepts relevant
     to the question.
  "conceptual_misunderstanding": bool,
  "conceptual_misunderstanding_expl": optional
    [string],

  // "wrong_core_problem_solving_strategy"
    should be true if the strategy chosen by
    the
  // agent ("core_problem_solving_strategy")
    was not suitable for answering the
    question.
  // For example, if the model decided to use
    curve fitting, when using a statistical
  // test would have been better.
  "wrong_core_problem_solving_strategy": bool,
  "wrong_core_problem_solving_strategy_expl":
  optional[string],

  // "wrong_method_within_strategy" should be
    true if the strategy chosen by the agent
  // ("core_problem_solving_strategy") was
    suitable for answering the question, but
  // failed due to the specific method chosen
    within the strategy. For example, the
    model
  // selected an incorrect statistical test,
    heuristic, threshold, or algorithmic
  // variant.
  "wrong_method_within_strategy": bool,
  "wrong_method_within_strategy_expl":
    optional[string],

  // "unsupported_assumption" should be true
    if for the method that the agent chose, it
  // made an unjustified assumption about the
    data or the options. For example, assuming
  // stationarity without evidence.
  "unsupported_assumption": bool,
  "unsupported_assumption_expl": optional[
    string],

  // "implementation_errors" should be true if
     there were coding or algorithmic mistakes
  // that affect the validity of the result.
    Code execution errors are that caused the
  // code to fail are not included in this
    category.
  "implementation_errors": bool,
  "implementation_errors_expl": optional[
    string],

  // "incorrect_result_interpretation" should
    be true if the agent incorrectly
    interprets
  // the result of any of his computations.
    For example, the model wrongly conclude
    that
  // the difference is statistically
    significant.
  "incorrect_result_interpretation": bool,
  "incorrect_result_interpretation_expl":
    optional[string],

  // "insufficient_evidence_guess" should be
    true if the agent answered whilst not
    having
  // gathered sufficiently convincing evidence.
```

```
  "insufficient_evidence_guess": bool,
  "insufficient_evidence_guess_expl": optional
    [string]
},

// This section is primarily meant for agents
    that had access to a code interpreter
// tool.
"code_problems": {
  // "code_result" should be one of the
    following options, indicating the overall
    success
  // of the code the model produced (if any):
  // - "success", if the agent managed to
    obtain the data it intended. If the code
  // initially failed due to errors, but the
    agent managed to fix the errors, it still
  // counts as a success.
  // - "partial failure", if the agent failed
    to obtain some of the desired data, but
  // managed to get at enough results to
    produce an answer. For example if the code
  // failed after printing sufficient evidence.

  // - "complete failure", if the agent failed
     to obtain the results and gave up or ran
  // out of turns.
  // - "no code", if the agent didn't produce
    any code.
  "code_result":
  enum["success", "partial failure", "complete
     failure", "no code"],
  "code_result_expl": optional[string],

  // "tool_usage_trouble" should be true if
    the agent had some trouble following
  // instructions to use the tool correctly.
    This mainly includes cases where the agent
  // generated an improper coding tool call or
     had trouble calling the tool properly.
  // Regular errors and exceptions that caused
     the code to fail are not included in this
  // category.
  "tool_usage_trouble": bool,
  "tool_usage_trouble_expl": optional[string],

  // "other" should be true if there was some
    other issue with the code that does not
  // fall into any of the other categories.
  "other": bool,
  "other_expl": optional[string]
},

"other_problems": {
  // "reasoning_answer_mismatch" should be
    true if the agent settled on one answer in
     the
  // thinking trace and then answered
    differently.
  "reasoning_answer_mismatch": bool,
  "reasoning_answer_mismatch_expl": optional[
    string],

  // "reasoning_tool_usage_mismatch" should be
     true if the agent claims it will use the
  // coding tool (such as saying "let's
    compute", or crafting a code solution in
    the
  // thinking trace) but then goes straight to
     answering.
  "reasoning_tool_usage_mismatch": bool,
  "reasoning_tool_usage_mismatch_expl":
    optional[string],

  // "hallucinated_values_in_reasoning" should
     be true if the agent introduces numerical
  // values that do not appear in the data or
    the code output. Reasonable rounding does
  // not count.
  "hallucinated_values_in_reasoning": bool,
  "hallucinated_values_in_reasoning_expl":
    optional[string]
}
},
```

Figure 8: Prompt for the LLM judge. The model receives the full conversation trace of the agent, including the question, the reasoning trace, the code (if applicable), and the final answer, along with the correct answer. The model is then asked to fill in a structured JSON questionnaire following the taxonomy described in Section 4.2. Note that we abbreviate `explanation` to `expl` in the field names for visualization purposes.

C Constructing the Error Taxonomy

We built an initial set of error categories by running two rounds of Gemini 3 Pro inference on a subset of incorrect `gpt-oss-120b` outputs. In the first round, Gemini 3 Pro received each incorrect trace paired with the correct answer and produced a short description of where the agent went wrong (Figure 9). In the second round, we fed all those descriptions into a single prompt asking Gemini to cluster them into fewer than ten categories and assign one to each entry (Figure 10).

Gemini returned seven preliminary categories:

1. **Skipping the Coding Phase:** The agent bypassed the coding tool or failed to execute planned code, relying on guesses instead.
2. **Statistical Misinterpretation & Methodological Errors:** Wrong statistical test, misread p-values or R^2, or failure to account for stationarity and distribution properties.
3. **Visual/Data Hallucination (Manual Inspection):** The agent described values and patterns from raw JSON without computing them.
4. **Conceptual & Semantic Misunderstandings:** Misunderstood core time series concepts (lag, amplitude, variance) or the specific definitions given in the problem.
5. **Coding Logic & Implementation Errors:** Buggy code, wrong library functions, or incorrectly implemented formulas.
6. **Flawed Heuristics & Rigid Thresholds:** Arbitrary numerical thresholds (e.g., 10^{-6} tolerance, 0.05 margin) producing incorrect binary decisions.

7. **Reasoning–Output Mismatch & Process Failures:** The agent ignored its own correct reasoning or code output when selecting the final answer.

These categories informed the failure mode section of the final taxonomy (Table 3). We expanded and restructured the section manually, adding the strategy and code problem categories.

```
You are given a conversation with a coding LLM
agent. In the conversation, the agent attempts
to answer a question by repeatedly producing
python code to programmatically query about
the relevant time series. The agent answered
wrong. The correct answer was "{data[correct]}".
Analyze where the error happened.
Answer in plain text. Do not produce more than
2 paragraphs.
=== Agent's Conversation ===
{data[conv]}
```

Figure 9: Prompt for per-trace error analysis. Each incorrect model output is analyzed individually.

```
{analysis 1}
—
(...)
—
{analysis n-1}
—
{analysis n}
======
Each chunk of text represents an analysis of
where an LLM coding agent made an error in
answering a question about time series. Cluster
all these errors into less than 10 error
categories. Then output a list assigning each
entry the error type. If an entry fits multiple
error types, select the most relevant one.
```

Figure 10: Prompt for clustering the per-trace descriptions into error categories.

D Selecting and Validating $\mathcal{L}_{\text{judge}}$

Development set. We build the development set from the configuration space $\mathcal{C} = \mathcal{D} \times \mathcal{S} \times \mathcal{M}$, where:

$$\mathcal{D} = \{\text{TIMESERIESEXAM}, \text{TSFU}\},$$
$$\mathcal{S} = \{\mathcal{M}_{\text{direct}}, \mathcal{M}_{\text{hybrid}}, \mathcal{M}_{\text{code}}\},$$
$$\mathcal{M} = \{\text{gpt-oss-120b}, \text{Qwen3-Next}\},$$

giving $|\mathcal{C}| = 12$. We sample 3 examples per configuration for 36 total. Each configuration's triple contains either 2 correct + 1 incorrect or 1 correct + 2 incorrect answers, chosen with equal probability, keeping the set roughly balanced. Since Qwen3-Next was not evaluated on the full TSFU dataset, the three TSFU × Qwen3-Next configurations were sampled from a separate pool of 60 randomly selected TSFU questions. No question appears more than once.

Judge configuration	Accuracy
`qwen3-next-80b` (single-phase)	86.5%
`qwen3-next-80b` (two-phase)	88.1%
`gpt-5.4-mini-2026-03-17`	89.3%
`gpt-oss-120b` (no system msg.)	89.3%
`gpt-oss-120b` (system msg.)	**90.3%**

Table 6: Agreement of each candidate judge configuration with manual labels on the 36-example development set.

Judge configurations. We test five judge setups on the development set, all applying the taxonomy from Section 4.2:

- **Qwen3-Next (single-phase):** the model generates the required JSON output directly. Constrained decoding prevents it from emitting reasoning tokens, so judgments lack explanations.
- **Qwen3-Next (two-phase):** the model first answers in free-form text, then rewrites its response into the JSON schema in a second turn, recovering reasoning at the cost of an additional inference call.
- **gpt-oss-120b (no system message):** the annotation task is specified entirely in the user message.
- **gpt-oss-120b (system message):** the same model with a custom system message priming it as an expert annotator.
- **gpt-5.4-mini:** an alternative commercial model, helps to better ground the judge capabilities.

Results. Table 6 reports agreement with manual labels for each configuration. `gpt-oss-120b` with a system message scores highest at 90.3%, while Qwen3-Next single-phase scores lowest at 86.5%.

The accuracy gap between the models is small overall. We select the best performing configuration `gpt-oss-120b` with a system message as $\mathcal{L}_{\text{judge}}$.

E Additional Results

Tables 7 and 8 break down accuracy by question category. Table 10 shows the distribution of strategy choices across domains. Table 11 shows the token usage statistics across all setups.

Table 9 reports inter-annotator agreement for each individual category.

category	$\mathcal{M}_{\text{code}}$	$\mathcal{M}_{\text{direct}}$	$\mathcal{M}_{\text{hybrid}}$
`gpt-oss-120b`			
similarity analysis	70.0%	65.0%	**75.8%**
pattern recognition	75.7%	69.1%	**80.9%**
causality analysis	65.3%	45.8%	**75.0%**
noise understanding	71.4%	76.2%	**81.0%**
anolmaly detection	55.6%	57.4%	**70.4%**
`qwen3-next-80b`			
similarity analysis	60.6%	65.8%	65.8%
pattern recognition	60.2%	66.0%	70.7%
causality analysis	55.9%	61.1%	68.1%
noise understanding	63.2%	67.9%	75.0%
anolmaly detection	35.5%	50.9%	56.5%

Table 7: Per-category accuracies on TIMESERIESEXAM for both models and all three setups. The best score for each category across both models and all three setups is in bold.

	`gpt-oss-120b`		
category	$\mathcal{M}_{\text{code}}$	$\mathcal{M}_{\text{direct}}$	$\mathcal{M}_{\text{hybrid}}$
fixed correlation	**79.5%**	72.2%	78.8%
lagged correlation	**42.5%**	30.0%	37.5%
anomalies	65.0%	74.0%	**75.0%**
seasonality	63.0%	56.0%	**72.5%**
fat tail	**86.5%**	69.0%	85.0%
stationarity	46.0%	28.0%	**51.5%**
structural break	44.0%	**52.0%**	49.5%
trend	**96.5%**	88.0%	**96.5%**
volatility	27.5%	15.0%	**31.0%**

Table 8: Per-category accuracies on TSFU for all three setups. Only `gpt-oss-120b` results are shown; `qwen3-next-80b` was not evaluated on the full TSFU dataset (see Section 3.4). The best score in each row is in bold.

Lastly, Table 12 illustrates three successful agent trajectories.

category	Cohen's κ
core reason for answer	1.00
core strategy	0.68
conceptual misunderstanding	0.22
wrong core problem solving strategy	0.41
wrong method within strategy	0.56
unsupported assumption	0.53
implementation errors	0.86
incorrect result interpretation	0.47
insufficient evidence guess	0.39
code result	0.72
tool usage trouble	0.82
reasoning answer mismatch	0.44
reasoning tool usage mismatch	0.59
hallucinated values in reasoning	0.60
overall	**0.57**

Table 9: Per-category Cohen's κ, indicating which categories are more difficult to distinguish reliably.

	simple arith.			stat. test			rolling stat.			curve fitting			spectral analysis			other			n
Domain	d	c	h	d	c	h	d	c	h	d	c	h	d	c	h	d	c	h	
TimeSeriesExam																			
Anomaly detection	67	36	63	0	10	7	0	12	6	0	9	13	0	9	8	33	25	4	638
Pattern recognition	80	36	42	0	18	17	0	3	4	0	24	24	0	10	10	20	8	3	2,171
Similarity	81	28	33	0	21	22	0	1	3	0	30	29	0	18	12	19	2	1	718
Noise understanding	69	26	32	0	40	40	0	12	10	1	10	10	0	10	8	30	4	1	504
Causality analysis	73	2	17	1	29	29	0	68	54	0	1	0	0	0	0	27	0	0	430
TSFU																			
Outliers	79	73	95	0	1	2	0	26	3	0	0	0	0	0	0	21	0	0	599
Trend	91	0	0	0	78	55	0	0	0	0	22	46	0	0	0	8	0	0	600
Statistical property	97	93	80	0	7	20	0	0	0	0	0	0	0	0	0	3	0	0	598
Correlation	96	25	15	0	75	85	0	0	0	0	0	0	0	0	0	4	0	0	1,198
Lagged correlation	86	3	20	0	83	64	0	14	15	0	0	0	0	0	0	14	0	0	597
Seasonality	72	16	18	0	0	0	0	0	2	0	0	0	0	84	78	28	0	1	599
Structural break	80	9	2	0	76	88	0	16	9	0	0	0	0	0	0	20	0	0	600
Stationarity	94	10	7	0	78	81	0	1	3	0	7	6	0	4	2	6	0	0	600
Volatility	82	0	0	0	0	0	2	100	100	0	0	0	0	0	0	16	0	0	600

Table 10: Distribution of core problem-solving strategies per problem domain, split by agent setup (d = $\mathcal{M}_{\text{direct}}$, c = $\mathcal{M}_{\text{code}}$, h = $\mathcal{M}_{\text{hybrid}}$). Each cell shows the percentage of answers within the given domain and setup, rounded to the nearest integer; within each setup, the dominant strategy is shown in bold. n = total number of judge-annotated answers in that domain across all models and setups.

Model	**Setup**	**Data**	**Mean**	**Median**	**Max**	**Cut off (%)**
`gpt-oss-120b`	$\mathcal{M}_{\text{direct}}$	TSE	13,340	9,933	30,040	26.8
`gpt-oss-120b`	$\mathcal{M}_{\text{code}}$	TSE	4,036	2,668	30,997	4.2
`gpt-oss-120b`	$\mathcal{M}_{\text{hybrid}}$	TSE	4,693	2,459	46,622	8.2
`gpt-oss-120b`	$\mathcal{M}_{\text{direct}}$	TSFU	7,900	3,825	30,172	5.8
`gpt-oss-120b`	$\mathcal{M}_{\text{code}}$	TSFU	4,335	2,737	48,540	<0.1
`gpt-oss-120b`	$\mathcal{M}_{\text{hybrid}}$	TSFU	3,719	2,178	43,251	0.1
`qwen3-next`	$\mathcal{M}_{\text{direct}}$	TSE	15,983	10,068	275,925	3.4
`qwen3-next`	$\mathcal{M}_{\text{code}}$	TSE	11,886	7,496	64,730	<0.1
`qwen3-next`	$\mathcal{M}_{\text{hybrid}}$	TSE	16,730	11,219	337,976	2.5

Table 11: Per-question output token statistics by model, setup, and dataset. Mean and median reflect the typical generation budget per question; the maximum captures the longest single conversation. "Cut off" is the percentage of questions whose answer was explicitly marked as truncated in the logged outputs. For $\mathcal{M}_{\text{code}}$ and $\mathcal{M}_{\text{hybrid}}$, token counts accumulate across all turns of the multi-turn conversation (code generation, execution feedback, and the final answer).

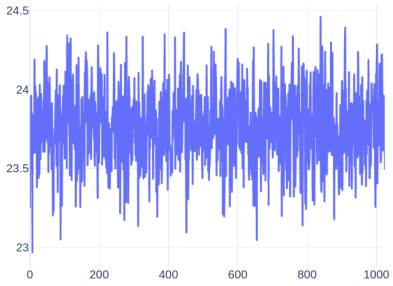
Example (a): $\mathcal{M}_{\text{direct}}$

What is the most likely mean of the given time series?

A) -14.69

B) 23.77

C) 8.87

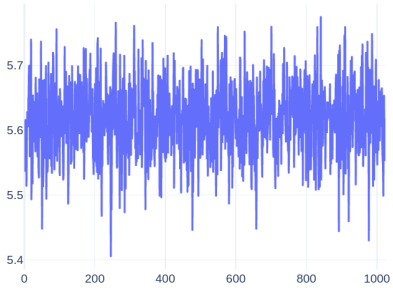


Example (b): $\mathcal{M}_{\text{code}}$

Is the mean stable over time in the given time series?

A) No

B)Yes



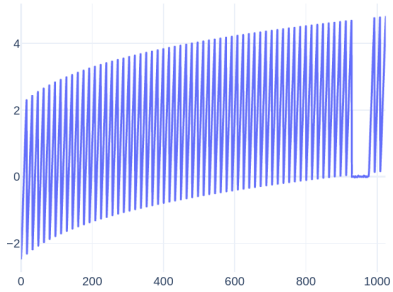
Example (c): $\mathcal{M}_{\text{hybrid}}$

The given time series has a trend and a cyclic component. It also has an anomaly. What is the most likely combination of components without the anomaly?

A) Exponential trend and square wave

B) Log trend and sawtooth wave

C) Linear trend and sine wave



Excerpts from the agent's output

[...]

First, I need to recall that the mean of a dataset is the average value. To find it, I would sum all the values and divide by the number of values. But since the data is provided as a JSON array under the "ts" key, I need to calculate the average of all those numbers.

[...]

Option A is -14.69. That seems way too low. All the data points are positive and around 23-24. A negative mean doesn't make sense here. So A is probably wrong.

Option C is 8.87. That's also way lower than the data points. Even the lowest values in the data are like 23.04, 23.13, 23.17, etc. So 8.87 is way too low. So C is likely incorrect.

That leaves option B) 23.77. That seems plausible. Let me check.

[...]

maybe the data is generated such that the mean is 23.77. Let's see, looking at some of the numbers, like 23.6, 23.7, 23.8, etc. If the average is 23.77, that's plausible. For example, if most values are between 23.5 and 24.0, the average would be in that range. The option B is 23.77, which is reasonable. The other options are way outside, so they can be eliminated.

Therefore, the correct answer is B.

[...]

We need to compute evidence: maybe test for stationarity of mean, e.g., using rolling mean, or statistical test like Augmented Dickey-Fuller (ADF) test for unit root. If the mean is stable, then the series is stationary in mean. The question likely expects a simple answer: Yes if mean stable, No otherwise.

[...]

```
# ADF test
adf_result = adfuller(series,
autolag='AIC')
adf_stat = adf_result[0]
adf_pvalue = adf_result[1]
```

[...]

ADF test statistic = -32.4668, p-value = 0.0 (very small). Critical values: -3.44 at 1% etc. Since test statistic is far less (more negative) than critical values, we reject null of unit root, indicating stationarity (mean stable).

Thus evidence suggests mean stable over time.

Thus answer: B) Yes.

[...]

Let's examine the increments: -2.46 to -2.15 (+0.31), -2.15 to -1.83 (+0.32), -1.83 to -1.51 (+0.32), -1.51 to -1.19 (+0.32), -1.19 to -0.876 (+0.314), -0.876 to -0.559 (+0.317)

[...]

Then after reaching 2.29, it resets to -2.32 (a drop). So the cyclic component is a sawtooth wave (linear increase then abrupt drop).

[...]

```
Linear fit within cycle coeffs:
[ 0.31716391 -2.46256645]
```

[...]

The residuals within cycle have std ~0.00075, which is extremely small. So within each cycle, the shape is essentially linear (i.e., a sawtooth wave).

[...]

```
log_idx = np.log(idx+1)
coeffs_log =
np.polyfit(log_idx, start_vals,
1)
print('Log fit coeffs (slope,
intercept):', coeffs_log)
```

[...]

The log fit has a slightly lower RMSE (0.140 vs 0.168) and higher correlation (0.978 vs 0.969). So the start values across cycles are better approximated by a log trend than a linear trend.

[...]

So B is the best.

Table 12: Three examples illustrating successful agent trajectories. (a) The agent finds the correct mean using simple value-range checks. (b) The agent correctly chooses to perform the Augmented Dickey-Fuller test, which provides evidence for stationarity. (c) The agent inspects the values and then separately tests for the sawtooth cyclic pattern and the trend.